



FRIEDRICH-ALEXANDER-UNIVERSITÄT
ERLANGEN-NÜRNBERG (FAU)

NHR@FAU

Assignment 0: Warming Up

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar

Student IdM: to37gulo

Student HPC-Account: ptfs356v

May 3, 2025

1. Numerical Integration Benchmark and Timing (25 Credits)

Goal

Approximate the integral

$$\int_0^1 \frac{1}{1+x} dx \approx \ln(2) \approx 0.69314718...$$

using the midpoint rule, and measure the runtime of the loop with high-resolution timing on the Fritz cluster.

Method

- Midpoint rule numerical integration.
- Runtime measured with `CLOCK_MONOTONIC`.
- Loop exits when runtime exceeds 10^{-7} seconds.
- Fixed CPU frequency using SLURM's `-cpu-freq` option.

Compiler and Execution Details

- **Compiler:** Intel icpx
- **Flags:** `-O3 -xHost`
- **Submission node:** fritz3
- **Compute node:** f1253
- **Run command:**

```
srtn --cpu-freq=2400000-2400000:performance ./Assignment0
```

Output

```
ln(2) 0.692835 with N = 10
Elapsed time: 1.32e-07 seconds
```

Code Snippet

```
double sum, x, delta_x, S, E;
int N = 10;
do {
    S = getTimestamp();
    delta_x = 1.0 / N;
    sum = 0.0;
    for (int i = 0; i < N; i++) {
        x = (i + 0.5) * delta_x;
        sum += 1.0 / (1.0 + x);
    }
    double ln = sum * delta_x;
    E = getTimestamp();
    if (E - S > 1e-7) {
        std::cout << "ln(2)  " << ln << " with N = " << N << std::endl;
        std::cout << "Elapsed time: " << (E - S) << " seconds" << std::endl;
    }
}
```

```
        break;
    }
    N *= 2;
} while (1);
```

Note

Attempts to fix the CPU frequency using `-cpu-freq=2400000-2400000` failed due to SLURM restrictions. As suggested in the discussion forum, the following alternative worked:

```
srunk --cpu-freq=2400000-2400000:performance ./Assignment0
```

The benchmark was therefore executed at a fixed clock frequency of 2.4 GHz using this approach.

2. CPU Cycles per Iteration and Timing Threshold (25 Credits)

Goal

Determine a reasonable minimum runtime to obtain stable and reproducible performance measurements, and compute the number of CPU cycles required per loop iteration.

Approach

I modified the threshold for minimum runtime by changing the condition in the benchmark from `1e-7` seconds to higher values, and observed the output, runtime, and per-iteration performance for each case.

All runs were executed on the Fritz compute node using a fixed CPU frequency of 2.4 GHz via:

```
srunk --cpu-freq=2400000-2400000:performance ./Assignment0
```

Results

Threshold = 1e-6 seconds

- $N = 1280$
- Elapsed time: 1.11×10^{-6} seconds
- Time per iteration: 8.67188×10^{-10} seconds
- Estimated CPU cycles per iteration: 2.08125

Threshold = 1e-5 seconds

- $N = 20480$
- Elapsed time: 1.715×10^{-5} seconds
- Time per iteration: 8.37402×10^{-10} seconds
- Estimated CPU cycles per iteration: 2.00977

Threshold = 1e-4 seconds

- $N = 163840$
- Elapsed time: 1.369×10^{-4} seconds
- Time per iteration: 8.35565×10^{-10} seconds
- Estimated CPU cycles per iteration: 2.00536

Analysis and Conclusion

From the results above, we observe that as the threshold increases:

- The per-iteration time becomes more stable.
- The estimated CPU cycles converge toward a consistent value around 2.0 cycles per iteration.

Given the relatively small number of cycles per iteration, measurements with thresholds below 10^{-6} seconds are noisy and unreliable. Therefore, a minimum threshold of 1×10^{-4} seconds is recommended to ensure accurate and reproducible performance analysis.

Assuming a fixed clock frequency of 2.4 GHz, the formula used to estimate CPU cycles per iteration was:

$$\text{Cycles per iteration} = \frac{\text{Elapsed Time}}{N} \times 2.4 \times 10^9$$

The result (2 cycles per iteration) is reasonable given that the loop body performs one floating-point division and one addition. The low cycle count also confirms the effectiveness of compiler optimization flags `-O3 -xHost`.

3. Performance Metrics and Discussion (20 Credits)

Performance Calculation

Assuming 5 floating-point operations per iteration (1 division, 2–3 additions, and 1 multiplication), the total FLOPs is:

$$\text{Total FLOPs} = 5 \times N$$

With $N = 163840$ and elapsed time $T = 1.369 \times 10^{-4}$ seconds, we calculate:

$$\text{FLOPs/s} = \frac{5 \times 163840}{1.369 \times 10^{-4}} \approx 5.98 \times 10^9 \text{ FLOPs/s}$$

Is FLOPs/s a Good Metric Here?

FLOPs/s is a common performance metric in HPC, but in this case, it is not ideal. This code performs only a small number of floating-point operations per iteration, with most time spent in a single division, which is slow and difficult to optimize.

As a result, the FLOPs/s metric gives an inflated sense of performance and does not reflect the efficiency or computational cost of the loop.

Alternative Metrics

More appropriate metrics include:

- **Cycles per iteration:** A direct hardware-level measure of how many CPU cycles are needed per loop iteration. In our case, this was consistently ≈ 2 cycles.
- **Time per iteration:** A useful latency-focused metric for small loops, which was $\approx 8.35 \times 10^{-10}$ seconds per iteration.
- **FLOPs per cycle:** Normalized performance — e.g., ≈ 2.5 FLOPs per cycle (if 5 FLOPs per iteration over 2 cycles).

These metrics better represent efficiency and cost on modern CPUs, especially for tight, low-FLOP kernels like this one.

In addition to standard FLOPs/s, the following metrics were calculated:

- **FLOPs per cycle:**

$$\frac{5 \text{ FLOPs per iteration}}{2.005 \text{ cycles}} \approx 2.49 \text{ FLOPs/cycle}$$

- **Iterations per second:**

$$\frac{1}{8.35565 \times 10^{-10}} \approx 1.197 \times 10^9 \text{ iterations/second}$$

- **Latency per operation:**

$$\frac{8.35565 \times 10^{-10}}{5} = 1.6711 \times 10^{-10} \text{ sec/op} = 0.1671 \text{ ns/op}$$

4. Turbo Mode and Effective Frequency Estimation (30 Credits)

Experiment Setup

To explore the effect of dynamic clock scaling, I reran the benchmark without fixing the CPU frequency. Instead, I used SLURM's performance governor to allow the CPU to boost to its maximum available frequency:

```
srunk --cpu-freq=performance ./Assignment0
```

Output

```
ln(2) 0.693147 with N = 327680
Elapsed time: 0.00018772 seconds
Time per iteration: 5.72876e-10 seconds
Estimated CPU cycles per iteration: 1.3749
```

Frequency Estimation

Assuming the performance scales linearly with clock speed, the actual frequency during execution can be estimated by comparing runtimes between fixed and turbo modes:

$$f_{\text{turbo}} = \frac{T_{\text{fixed}}}{T_{\text{turbo}}} \times 2.4 \text{ GHz}$$

Using:

- $T_{\text{fixed}} = 0.000136899$ seconds
- $T_{\text{turbo}} = 0.00018772$ seconds

$$f_{\text{turbo}} = \frac{0.000136899}{0.00018772} \times 2.4 \approx 1.75 \text{ GHz}$$

Discussion

Surprisingly, the performance governor run was slower than the fixed-frequency run. This suggests one of the following:

- The Fritz cluster may not support Turbo Boost, or it may be disabled for user jobs.
- CPU cores may have been thermally throttled or in power-saving mode.

SLURM Job Script Setup

To automate the execution of the benchmark for both fixed-frequency and turbo-mode configurations, the following SLURM job script was created and submitted using `sbatch`:

```
#!/bin/bash -l
#
#SBATCH --nodes=1
#SBATCH --time=00:08:00
#SBATCH --job-name=ln2
#SBATCH --export=NONE
#SBATCH --output=ln2_output.txt

# First non-empty non-comment line ends SBATCH options
unset SLURM_EXPORT_ENV

# Load Intel compiler
module load intel

# Compile the benchmark code
icpx -O3 -xHost Assignment0.cpp -o Assignment0

# Run with fixed CPU frequency
echo "Running with fixed CPU frequency (2.4 GHz)..."
srun --cpu-freq=2400000-2400000:performance ./Assignment0

# Run with turbo (performance governor)
echo "Running with performance governor..."
srun --cpu-freq=performance ./Assignment0
```

Listing 1: SLURM Job Script: `run_job_assignment0.sh`

This script ensures reproducibility by compiling the code within the job environment, and uses `srun` with the appropriate `-cpu-freq` options to differentiate between fixed and dynamic frequency scaling. Output is automatically saved to `ln2_output.txt`.

Appendix

Terminal Output for Point 1 – Numerical Integration Benchmark and Timing

```
ptfs356v@f1253:~$ icpx -o3 Assignment0.cpp -o Assignment0
ptfs356v@f1253:~$ srun --cpu-freq=2400000-2400000:performance ./Assignment0
ln(2) ≈ 0.692835 with N = 10
Elapsed time: 1.32e-07 seconds
ptfs356v@f1253:~$
```

Figure 1: Terminal output showing compilation, fixed CPU frequency execution, and runtime for Point 1.

Terminal Output for Point 2 – Cycle and Iteration Analysis

```
ptfs356v@f1253:~$ icpx -o3 Assignment0.cpp -o Assignment0
ptfs356v@f1253:~$ srun --cpu-freq=2400000-2400000:performance ./Assignment0
ln(2) ≈ 0.693147 with N = 1280
Elapsed time: 1.11e-06 seconds
Time per iteration: 8.67188e-10 seconds
Estimated CPU cycles per iteration: 2.08125
ptfs356v@f1253:~$ icpx -o3 Assignment0.cpp -o Assignment0
ptfs356v@f1253:~$ srun --cpu-freq=2400000-2400000:performance ./Assignment0
ln(2) ≈ 0.693147 with N = 20480
Elapsed time: 1.715e-05 seconds
Time per iteration: 8.37402e-10 seconds
Estimated CPU cycles per iteration: 2.00977
ptfs356v@f1253:~$ icpx -o3 Assignment0.cpp -o Assignment0
ptfs356v@f1253:~$ srun --cpu-freq=2400000-2400000:performance ./Assignment0
ln(2) ≈ 0.693147 with N = 163840
Elapsed time: 0.000136899 seconds
Time per iteration: 8.35565e-10 seconds
Estimated CPU cycles per iteration: 2.00536
ptfs356v@f1253:~$
```

Figure 2: Terminal output showing calculation of time per iteration and estimated CPU cycles at a 1e-6, 1e-5 and 1e-4 seconds threshold respectively.

Terminal Output for Point 4 – Turbo Mode Test

```
ptfs356v@f1253:~$ srun --cpu-freq=performance ./Assignment0
ln(2) ≈ 0.693147 with N = 327680
Elapsed time: 0.00018772 seconds
Time per iteration: 5.72876e-10 seconds
Estimated CPU cycles per iteration: 1.3749
```

Figure 3: Terminal output from performance governor run (no fixed frequency), showing the loop runtime, time per iteration, and estimated CPU cycles per iteration.